

Wed, Apr 8 at 5:01 PM



Pasted markdown.md

File

I'm locking this down into a practical build path: HDA interface first, then the four internal masks, then shader wiring, then AOV extraction and TOPs readiness. I'm also checking current SideFX docs for the Solaris/Karma pass side so I do not hand you stale node advice.

I've got the renderer side verified enough to avoid dead advice: custom shader signals can go through Karma AOV nodes, but they still need matching render vars in Solaris, and Karma Render Settings is the node that owns those render vars/render products. I'm folding that into a stripped-down HDA design instead of giving you a generic MaterialX lecture.

Thought for 3m 14s >

Your architecture is valid. I would keep it: one hero mesh, one master neuromat HDA, JSON-driven parameters, UV-stage texturing, baked AO/curvature support, MaterialX shading, and TOPs later for scale. That is good enough for Phase 1, and it does not need a triplanar redesign right now.  Pasted markdown

The one real problem is this: **do not generate the final training dataset from Houdini Apprentice**. SideFX's current docs say Apprentice is limited to **1920×1080** and renders to file formats other than `.picnc` are **watermarked**. That is poison for Phase 2. Build and

debug neuromat in Apprentice if you want, but the final EXR dataset needs Indie, Education, or commercial Houdini.  SideFX +2

1. Minimal neuromat interface

Do **not** promote the whole JSON schema as visible UI. That will turn the HDA into garbage.

Promote only this:

Source

- json_path
- material_id
- reload_json

Support inputs

- ao_source_mode (primvar / texture)
- curvature_source_mode (primvar / texture)
- ao_texture_path (optional)
- convex_texture_path (optional)
- concave_texture_path (optional)

Lookdev / debug

- preview_mode (beauty / basecolor / roughness / variation / dirt / wear / l
- enable_debug_aovs
- force_seed_override
- seed_override

Safety clamps

- max_variation_color_amount
- max_variation_roughness_amount
- max_dirt_amount
- max_wear_amount
- max_bump_amount

Everything else stays internal as root attributes or internal parameters loaded from JSON:

- all shader_parameters
- all procedural_parameters
- metadata strings

That keeps batch rendering stable and prevents hand-edited drift.

2. First fix inside the HDA: normalize support signals

Before you build dirt and wear, make the support signals sane.

You need three normalized masks inside neuromat :

```
ao_occluded    : 0..1, where 1 = cavity / sheltered  
convex_mask    : 0..1, where 1 = exposed edge / peak  
concave_mask   : 0..1, where 1 = recess / valley
```



Do not keep "raw curvature" as a vague thing. Split it once and never think about it again.

Best practical approach:

- if your baked AO/curvature are already on points and working, keep them
- but convert/remap them into stable 0..1 signals near the top of the HDA
- if MaterialX access to those attrs becomes annoying, bake them once to UV textures and freeze them

That is not a redesign. That is cleanup.

3. Where each mask should live

Keep the split simple.

COPS

- VARIATION base texture
- optional low-frequency dirt breakup
- optional UV-space debug maps you want to inspect directly

MaterialX

- remaps
- combines
- branch-specific micro breakup
- final bump height
- final shader wiring
- AOV taps

That gives you inspectable masks without stuffing the whole logic into COPs or the whole mess into MaterialX.

4. Internal mask logic

VARIATION

Keep it weak. It is not dirt and not wear.

Use it only for:

- slight base value modulation
- slight saturation modulation
- slight roughness breakup

Recommended behavior:

```
variation_mask = centered_low_frequency_noise(seed, noise_scale)
variation_color_gain = 1.0 + variation_mask * 0.02..0.06
variation_roughness_delta = variation_mask * 0.01..0.04
```



Rules:

- no hue swings unless explicitly authored
- no high-contrast blotches
- no edge emphasis
- clamp hard

DIRT

Dirt should be cavity-biased and broad.

Use this structure:

```
dirty_base =  
    ao_occluded  
    * low_frequency_noise(seed + 11, noise_scale * 0.5)  
    * optional_concave_bias  
  
dirty_mask =  
    clamp(  
        smoothstep(dirty_low, dirty_high, dirty_base)  
        * dirty_parameter,  
        0,  
        1  
    )
```



What dirt should affect:

- roughness: strongly
- base color: slightly toward dust/grey/tint
- maybe transmission color very slightly for translucent materials
- not metalness
- not IOR
- not transmission depth in any strong way

So:

```
base_color_after_dirt =  
    mix(base_color_after_variation, dirt_tinted_color, dirt_mask * 0.1..0.25)  
  
roughness_after_dirt =  
    base_roughness + dirt_mask * 0.08..0.30
```



WEAR

Wear should be convex-biased and sharper than dirt.

Use this structure:

```
wear_base =  
    convex_mask  
    * medium_frequency_noise(seed + 29, noise_scale)  
    * optional_directional_or_scratch_pattern  
  
wear_mask =  
    clamp(  
        smoothstep(wear_low, wear_high, wear_base)  
        * wear_parameter,  
        0,  
        1  
    )
```



What wear should affect depends on family:

Metals

- can slightly brighten exposed areas
- can either reduce roughness on polished edge wear
- or increase roughness on scratched wear

Dielectrics / rubber

- usually slight desaturation or value shift
- usually roughness increase, not fake exposed metal

Translucent

- keep wear subtle
- mostly roughness change, almost no big color event

Do not use one universal wear response for all families.

BUMP

Bump must stay in "highlight-breakup" territory.

Branch it by `bump_type` :

stochastic

- fine fractal / irregular micro-noise

directional

- stretched noise / brushed grain / aligned scratch height
- aligned in UV or object/tangent space
- never camera space

cellular

- crater / hammered / pitted structure
- low amplitude unless explicitly heavy material

none

- flat height = 0

Use:

```
bump_height = branch_height(seed, noise_scale, bump_type) * bump_scale
```



Then clamp it hard.

Your goal is:

- highlight change
- reflection breakup
- not silhouette deformation
- not fake displacement

5. Final shader wiring

Use one `Mt1X Standard Surface` unless you hit a real limitation.

Wire it like this:

```
base_color_input
-> JSON base_color
-> variation modulation
-> dirt tint modulation
-> optional wear tint modulation
```

```
specular_roughness_input
-> JSON roughness
-> variation roughness delta
-> dirt roughness increase
-> wear roughness change
```

```
metalness / specular_ior / k / transmission / thin_walled / coat
-> direct JSON control only
```



```
normal / bump  
-> bump branch result only
```

Do **not** procedurally randomize:

- metalness
- IOR
- k
- transmission depth
- thin walled
- coat presence

That belongs to authored identity, not procedural noise.

6. Family response rules

You need one internal family switch driven from `metadata.category` or `base`.

Use only 3 response families for now:

```
metal  
dielectric  
translucent
```



That is enough.

The family switch only changes:

- dirt tint strength
- wear color response
- wear roughness response
- bump amplitude clamp

Do not make separate giant shader trees per family.

7. AOV plan

This is the clean split:

Standard / render-side AOVs

Use Karma render vars for:

- beauty
- alpha
- depth

Karma Render Settings creates the render settings prim, render product, and render vars. Karma's docs also note that custom shading signals marked with the Karma AOV VOP still need matching render vars in Solaris. Standard Render Vars gives you predefined AOV/image-plane setups.  SideFX +3

Material-authored AOVs

Use **Karma AOV VOP** nodes inside `neuromat` for:

- `basecolor_raw`
- `roughness_raw`
- `variation_raw`
- `dirt_raw`
- `wear_raw`
- `bump_height_raw`
- `ao_raw`
- `convex_raw`
- `concave_raw`

The AOV node passes the signal through unchanged, and the name/type must match the render var definition in Solaris. Use `Raw` source type and `float` / `color3f` formats as

appropriate.  SideFX

Normals and position

For your use case, do **not** trust a generic “normal” pass blindly. Output them explicitly.

Make custom AOVs:

- `N_world_raw`
- `P_world_raw`

That way you know exactly what space you are training on.

Base color

H21 Karma also has better support for base color AOV export on `Mt1X Standard Surface`, and Karma Render Settings has base-color-oriented presets.  SideFX

8. Render format

Use EXR.

My recommendation:

- beauty, base color: half is fine
- masks, roughness, AO, wear, dirt: float if storage is acceptable, otherwise half
- `P`, `N`, `Pz` : full float

Karma Standard Render Vars supports 16-bit or 32-bit precision per AOV.  SideFX

9. Determinism rules

These matter more than shader cleverness.

Never let the HDA depend on:

- time
- frame number
- viewport state
- camera position for procedural generation
- hidden random nodes with changing seeds

Use one deterministic seed system:

```
variation_seed
variation_seed + 11 -> dirt
variation_seed + 29 -> wear
variation_seed + 53 -> bump
```



No floating randomness. No “randomize each cook”.

10. TOPs readiness

Do not wedge live shader noise parameters directly in TOPs.

Instead:

1. generate the 2,000 JSON material entries first
2. each TOP work item picks one `material_id`
3. a second axis is fixed `camera_id`
4. render path is deterministic:
 - `/ {material_id} / {camera_id} / beauty.exr`
 - `/ {material_id} / {camera_id} / basecolor.exr`
 - and so on
5. write a manifest row per frame with:
 - material id
 - semantic text

- camera id
- camera transform
- intrinsics
- all output file paths

TOPs should orchestrate. JSON should define material identity.

11. Build order I would use

1. Normalize AO / convex / concave signals.
2. Finish VARIATION and lock its amplitude down.
3. Build DIRT .
4. Build WEAR .
5. Build BUMP branches.
6. Wire one stable MtlX Standard Surface .
7. Add debug preview enum.
8. Add custom Karma AOV taps.
9. Render 20 materials × 8 cameras.
10. Kill anything that looks too strong or unstable.
11. Then scale.

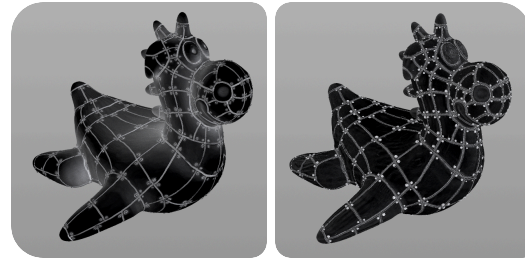
The shortest correct path is this: **keep the architecture, simplify the mask math, export raw supervision AOVs, and do not render the final dataset from Apprentice.**

Next useful step is an exact inside-the-HDA node graph: subnet by subnet, with the MaterialX nodes and remap logic laid out in order.

Sources

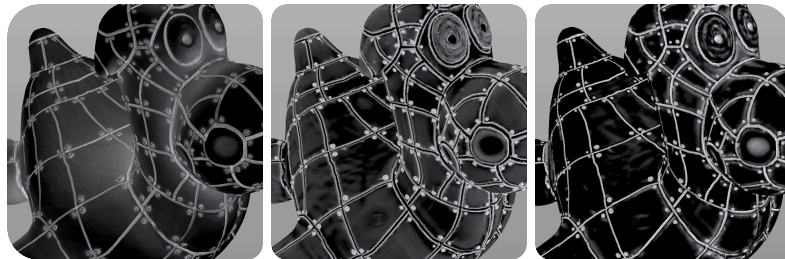


Fri, Apr 10 at 2:54 PM



Ok, let me handle "Normalize AO / convex / concave signals". Here is AO, it is in a range 0-1 (see attached image1). How can I get convex_mask and concave_mask? I can get curvature (image2) with the Measure SOP within the range 0-28, IDK how to split it and normalize it correctly.

Fri, Apr 10 at 10:56 PM



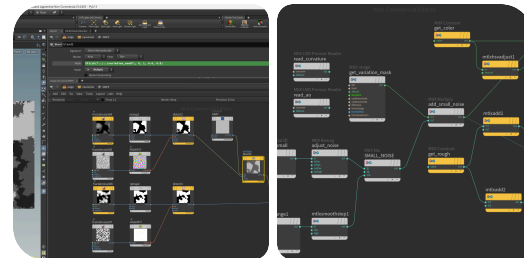
AO, convex (convex_reference = 0.2), concave
(concave_reference = 0.1). Does that look close to what we
need?



Exploring concave. Attached image 1 is a low-resolution Houdini Ruber Toy, and image 2 is the same object sculpted. Same values (concave reference = 0.3). On low res model I can clearly see the concave map nature, while on sculpted everything is shifted to seams/rivets. Should I drop the sculpted model and work with low res, or should I tweak the sculpted to get a feeling closer to low-res?

Image 3 is concave, reference = 0.03, which seems to look closer to low res.

Tue, Apr 14 at 1:03 PM

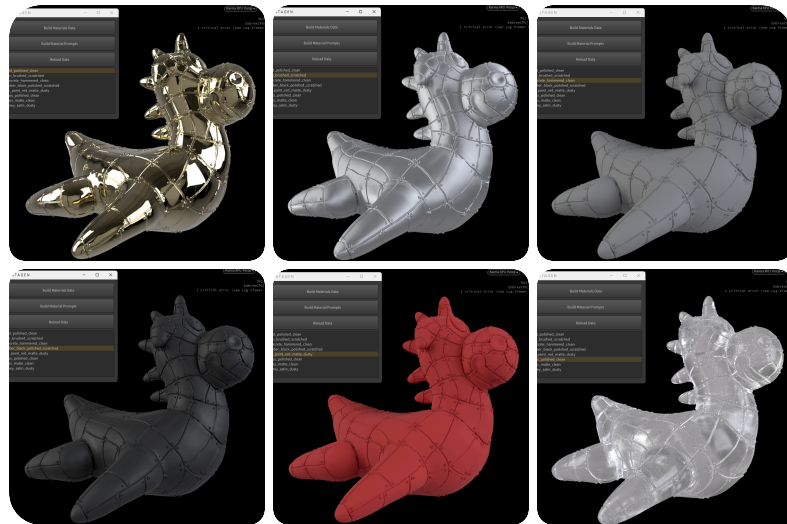


Ok, got my AO/convex/concave. Let's move to step 2, "Finish VARIATION and lock its amplitude down."

Attached image 1 is VARIATION_MASK COP network. Using multiply to mix 2 noises, big and medium. The small noise added in Material X network. The image 2 is Material X network.

I am good with the look-and-feel of VARIATION_MASK. Should we make sure it's values in the correct ranges?

Thu, Apr 16 at 1:25 AM



Let's do BUMP?

Thought for 6m 46s >

Yes. Build `BUMP` almost entirely in **MaterialX**, not in COPS. Your brief defines bump as a procedural micro-surface effect driven by `bump_scale`, `bump_type`, `noise_scale`, and `variation_seed`, and the goal is believable highlight/reflection breakup, not displacement. MaterialX in Solaris is the correct place to build that shader logic for Karma.

 Pasted markdown  SideFX +1

0. First decision: what to build in COPS vs MaterialX

For **BUMP V1**, create **no bump maps** in COPS.

Keep COPS for:

- `VARIATION_MASK`
- `DIRT_NOISE_MASK`
- `WEAR_NOISE_MASK`

Build **all four bump branches** in the **MaterialX network**:

- `none`
- `stochastic`
- `directional`

- `cellular`

Reason: bump is better as a live procedural scalar-height signal inside the shader, and MaterialX already gives you the nodes you need for coordinates, noise, remap, switching, and height-to-normal conversion. `Mt1X Position`, `Mt1X Texcoord`, `Mt1X Fractal3D`, `Mt1X Unified Noise 2D/3D`, `Mt1X Range`, `Mt1X Switch`, and `Mt1X Heighttonormal` are all there for this.  SideFX +7

1. Do not switch on the string inside MaterialX

Do **not** try to read the JSON string `"bump_type"` directly inside the MaterialX graph and branch on it.

Use this instead:

- keep the original JSON string `bump_type` for readability
- create a second integer control: `bump_mode`
- map it like this:
 - `none = 0`
 - `stochastic = 1`
 - `directional = 2`
 - `cellular = 3`

This is the correct move because `Mt1X Switch` selects by the numeric `which` parameter, not by a string name. SideFX's docs say the output comes from `floor(which)+1`, clamped to the available inputs. MaterialX USD Primvar Reader can read typed primvars, but the signature must match the primvar type exactly, so string-reading is not the hard part; **string-based branch switching** is the bad part.  SideFX +1

Use a Python mapping in your JSON loader or HDA callback:

</> Python



▶ Run

```
bump_type_to_mode = {  
    "none": 0,  
    "stochastic": 1,  
    "directional": 2,  
    "cellular": 3,  
}  
  
bump_type_string = material_data["procedural_parameters"]["bump_type"]  
bump_mode_integer = bump_type_to_mode.get(bump_type_string, 1)  
  
houdini_node.parm("bump_mode").set(bump_mode_integer)  
houdini_node.parm("bump_type_label").set(bump_type_string)
```

bump_type_label is optional. It is just for debugging.

2. Where to build it

Build this inside your **Karma Material Builder / MaterialX graph**, not in the **MAPS Copernicus** context.

Inside the material, create one network box or subnet called:

BUMP



Inside **BUMP** , create these sections:

```
BUMP_SHARED  
BUMP_NONE  
BUMP_STOCHASTIC  
BUMP_DIRECTIONAL  
BUMP_CELLULAR
```



BUMP_SELECT

BUMP_APPLY

That keeps it readable and lets you debug branch by branch.

3. Parameters to add on the material / HDA

Add these promoted controls on the material/HDA:

```
bump_mode      integer menu    (0 none, 1 stochastic, 2 directional, 3 cellular)
bump_scale     float
noise_scale    float
variation_seed float or integer
bump_cap       float
```



Start values:

```
bump_scale  = 0.01
noise_scale = 1.0
variation_seed = 0
bump_cap    = 0.02
```



For polished clean materials, use a smaller bump_cap , around:

```
0.006
```



4. Shared nodes to create first

Inside BUMP_SHARED , create these nodes.

4.1 Scalar parameter nodes

Use ordinary material parameters or constant nodes wired from promoted parms:

```
get_bump_scale
get_noise_scale
get_variation_seed
get_bump_cap
```



Use float type for all of them except `bump_mode` , which is integer.

4.2 Seed offsets

You need stable, nonzero offsets so all branches do not line up the same way.

Create:

```
seed_offset_3d  MtlX Constant  vector3
seed_offset_2d  MtlX Constant  vector2
```



Set them with parameter expressions derived from `variation_seed` , for example:

```
seed_offset_3d = (
    variation_seed * 17.13,
    variation_seed * 9.71,
    variation_seed * 3.19
)

seed_offset_2d = (
    variation_seed * 17.13,
    variation_seed * 9.71
)
```



You are just creating deterministic coordinate offsets.

4.3 Coordinate sources

Create:

```
object_position    MtlX Position    Space = object
uv_coords          MtlX Texcoord    index = 0
```



MtlX Position can output object/model/world-space coordinates, and MtlX Texcoord gives you the current UV set. Use object space for isotropic 3D bump, and UV space for directional grain.  SideFX +1

5. Branch 1: BUMP_NONE

This branch is trivial.

Create:

```
bump_none_value    MtlX Constant    float = 0
```



That is the whole branch.

6. Branch 2: BUMP_STOCHASTIC

This is your default micro-breakup branch.

6.1 Nodes to create

Create these nodes:

```
stochastic_frequency_coarse    MtlX Constant    vector3
stochastic_frequency_fine      MtlX Constant    vector3

stochastic_position_coarse     MtlX Multiply
stochastic_position_fine       MtlX Multiply

stochastic_position_coarse_seeded    MtlX Add
stochastic_position_fine_seeded      MtlX Add
```



stochastic_noise_coarse	Mt1X Fractal3D
stochastic_noise_fine	Mt1X Fractal3D
stochastic_coarse_weight	Mt1X Multiply
stochastic_fine_weight	Mt1X Multiply
stochastic_sum	Mt1X Add

6.2 Parameter values

Set:

```
stochastic_frequency_coarse = (
    noise_scale * 6.0,
    noise_scale * 6.0,
    noise_scale * 6.0
)

stochastic_frequency_fine = (
    noise_scale * 24.0,
    noise_scale * 24.0,
    noise_scale * 24.0
)
```



Set the two Mt1X Fractal3D nodes roughly like this:

For stochastic_noise_coarse :

```
amplitude  = 1.0
octaves    = 3
lacunarity = 2.0
diminish   = 0.5
```



For stochastic_noise_fine :

```
amplitude  = 1.0
octaves    = 2
lacunarity = 2.0
diminish   = 0.5
```



Mt1X Fractal3D is already **zero-centered** noise, which is exactly what you want for a height signal.  SideFX

6.3 Connections

Wire them exactly like this:

```
object_position
  -> stochastic_position_coarse.in1
stochastic_frequency_coarse
  -> stochastic_position_coarse.in2

stochastic_position_coarse.out
  -> stochastic_position_coarse_seeded.in1
seed_offset_3d
  -> stochastic_position_coarse_seeded.in2

stochastic_position_coarse_seeded.out
  -> stochastic_noise_coarse.position
```



Then duplicate the same for the fine branch:

```
object_position
  -> stochastic_position_fine.in1
stochastic_frequency_fine
  -> stochastic_position_fine.in2

stochastic_position_fine.out
  -> stochastic_position_fine_seeded.in1
```



```
seed_offset_3d
-> stochastic_position_fine_seeded.in2

stochastic_position_fine_seeded.out
-> stochastic_noise_fine.position
```

Now weight and add them:

```
stochastic_noise_coarse.out
-> stochastic_coarse_weight.in1
0.7
-> stochastic_coarse_weight.in2

stochastic_noise_fine.out
-> stochastic_fine_weight.in1
0.3
-> stochastic_fine_weight.in2

stochastic_coarse_weight.out
-> stochastic_sum.in1
stochastic_fine_weight.out
-> stochastic_sum.in2
```



stochastic_sum.out is your **stochastic branch output**.

7. Branch 3: BUMP_DIRECTIONAL

This is for brushed / aligned grain.

7.1 Nodes to create

Create:

```
directional_frequency    MtlX Constant    vector2
directional_uv_scaled    MtlX Multiply
```



directional_uv_seeded	Mt1X Add
directional_noise	Mt1X Unified Noise 2D
directional_range	Mt1X Range
directional_breakup_weight	Mt1X Multiply
stochastic_breakup_weight	Mt1X Multiply
directional_sum	Mt1X Add

7.2 Parameter values

Set frequency like this:

```
directional_frequency = (
    noise_scale * 80.0,
    noise_scale * 8.0
)
```



This makes the pattern vary much faster in one UV direction than the other, which creates elongated grain. If the grain is rotated the wrong way, swap the two values. `Mt1X Texcoord` provides the UV coordinates, and `Mt1X Unified Noise 2D` can generate Fractal, Cell, or Worley based patterns in 2D.  SideFX +1

Set `directional_noise` :

```
Noise Type = Fractal
```



Set `directional_range` :

```
inlow    = 0
inhigh   = 1
outlow   = -1
```



```
outhigh = 1
doclamp = on
```

Mt1X Range is the clean way to remap a $0..1$ signal into $-1..1$.  SideFX

7.3 Connections

```
uv_coords
  -> directional_uv_scaled.in1
directional_frequency
  -> directional_uv_scaled.in2

directional_uv_scaled.out
  -> directional_uv_seeded.in1
seed_offset_2d
  -> directional_uv_seeded.in2

directional_uv_seeded.out
  -> directional_noise.texcoord

directional_noise.out
  -> directional_range.in
```



Now mix in weak stochastic breakup so it does not look like perfect synthetic stripes:

```
directional_range.out
  -> directional_breakup_weight.in1
0.8
  -> directional_breakup_weight.in2

stochastic_sum.out
  -> stochastic_breakup_weight.in1
0.2
  -> stochastic_breakup_weight.in2
```



```
directional_breakup_weight.out
-> directional_sum.in1
stochastic_breakup_weight.out
-> directional_sum.in2
```

`directional_sum.out` is your **directional branch output**.

8. Branch 4: BUMP_CELLULAR

This is for hammered / pitted / crater-like bump.

8.1 Nodes to create

Create:

<code>cellular_frequency</code>	MtlX Constant	vector3
<code>cellular_position_scaled</code>	MtlX Multiply	
<code>cellular_position_seeded</code>	MtlX Add	
<code>cellular_noise</code>	MtlX Unified Noise 3D	
<code>cellular_range</code>	MtlX Range	
<code>cellular_weight</code>	MtlX Multiply	
<code>cellular_breakup_weight</code>	MtlX Multiply	
<code>cellular_sum</code>	MtlX Add	



8.2 Parameter values

Set frequency:

```
cellular_frequency = (
    noise_scale * 18.0,
    noise_scale * 18.0,
    noise_scale * 18.0
)
```



Set cellular_noise :

Noise Type = Worley



If cell gives a better hammered feel in your build, use cell instead. Mt1X Unified Noise 3D is specifically meant to give you those noise families from one node.  SideFX

Set cellular_range :

```
inlow    = 0
inhigh   = 1
outlow   = -1
outhigh  = 1
doclamp  = on
```



8.3 Connections

```
object_position
-> cellular_position_scaled.in1
cellular_frequency
-> cellular_position_scaled.in2

cellular_position_scaled.out
-> cellular_position_seeded.in1
seed_offset_3d
-> cellular_position_seeded.in2

cellular_position_seeded.out
-> cellular_noise.position

cellular_noise.out
-> cellular_range.in
```



Now add weak stochastic breakup:

```
cellular_range.out
    -> cellular_weight.in1
0.85
    -> cellular_weight.in2

stochastic_sum.out
    -> cellular_breakup_weight.in1
0.15
    -> cellular_breakup_weight.in2

cellular_weight.out
    -> cellular_sum.in1
cellular_breakup_weight.out
    -> cellular_sum.in2
```

cellular_sum.out is your **cellular branch output**.

9. Create the switch

Now make one selector node.

Create:

```
bump_branch_switch    MtlX Switch    output type = float
```

Connect inputs like this:

```
in1 = bump_none_value.out
in2 = stochastic_sum.out
in3 = directional_sum.out
in4 = cellular_sum.out
```

Set the `which` parameter on the switch to the material/HDA integer parameter `bump_mode` .

If the switch is one level below the promoted parameter, the expression is usually something like:

```
ch("../bump_mode")
```



or

```
ch("../../bump_mode")
```



depending on your nesting level.

MtlX Switch selects input `floor(which)+1` , so this mapping is correct:

- 0 -> in1 -> none
- 1 -> in2 -> stochastic
- 2 -> in3 -> directional
- 3 -> in4 -> cellular  SideFX

10. Apply amplitude and clamp

After the switch, create:

bump_scaled	MtlX Multiply
bump_clamped	MtlX Clamp
height_to_normal	MtlX Heighttonormal



10.1 Connections

```

bump_branch_switch.out
  -> bump_scaled.in1
get_bump_scale
  -> bump_scaled.in2

```



Then clamp it:

```

bump_scaled.out
  -> bump_clamped.in
-bump_cap
  -> bump_clamped.low
bump_cap
  -> bump_clamped.high

```



Then convert scalar height to a normal vector:

```

bump_clamped.out
  -> height_to_normal.in
1.0
  -> height_to_normal.scale

```



Finally:

```

height_to_normal.out
  -> mtlxstandard_surface.normal

```



Mt1X Heighttonormal converts a scalar height map to a vector3 normal map and has a dedicated `scale` parameter. This is cleaner than trying to fake bump through roughness.

Mt1X Standard Surface takes a normal input.  SideFX +1

11. Optional debug outputs

Add these now:

```
bump_branch_raw
bump_height_final
```



For the raw branch output, tap:

```
bump_branch_switch.out
```



For the final height used in shading, tap:

```
bump_clamped.out
```



Send both to Karma AOV nodes later if you want to debug them.

12. What not to do

Do not do these:

- do not send raw 0..1 cellular or directional noise straight into the normal input
- do not switch on the string inside MaterialX
- do not build separate COPS bump maps for every branch
- do not let glass and polished metals use the same bump amplitude as hammered concrete
- do not use displacement for this stage

13. The exact minimal production version

If you want the shortest usable build, do this:

```
BUMP_NONE :
    constant 0
```



```
BUMP_STOCHASTIC:
    object_position
    -> scale
    -> add seed
    -> fractal3d coarse
    +
    object_position
    -> scale finer
    -> add seed
    -> fractal3d fine
    -> weighted add

BUMP_DIRECTIONAL:
    texcoord
    -> anisotropic scale
    -> add seed
    -> unified noise 2d (fractal)
    -> range 0..1 to -1..1
    -> add weak stochastic

BUMP_CELLULAR:
    object_position
    -> scale
    -> add seed
    -> unified noise 3d (worley/cell)
    -> range 0..1 to -1..1
    -> add weak stochastic

MtlX Switch(which = bump_mode)
    -> Multiply by bump_scale
    -> Clamp to +/- bump_cap
    -> Heighttonormal
    -> standard_surface.normal
```

That is the clean V1.

14. My blunt recommendation for your current project

Use this exact policy:

- **COPS:** no bump maps
- **MaterialX:** all bump logic
- **branch selection:** integer `bump_mode`
- **keep** `bump_type` **string only** for JSON readability
- **switch by integer**
- **use** `Heighttonormal` , **not displacement**

That is the shortest correct path.

If you want, next I'll give you the same thing as a strict Houdini node checklist with exact node names in creation order, one line per node.

Sources



Fri, Apr 24 at 1:17 PM

We don't have "bump_cap" value defined in our JSON.
Should we update material generation script to introduce it or we can define it in Houdini Material X network?

Thought for 7s >

Do **not** add `bump_cap` to JSON right now.

Define it **inside the Houdini MaterialX/HDA logic** as an **internal safety clamp**.

That is the correct choice for your current system.

Why

`bump_cap` is not really a material identity parameter.
It is a **shader safety rail**.

Your JSON should keep:

- material identity
- intended artistic controls

Like:

- `bump_type`
- `bump_scale`
- `noise_scale`
- `variation_seed`

But `bump_cap` is more like:

- "never let this branch go insane"
- "polished clean glass must stay nearly flat"
- "hammered concrete can go further"

That is **implementation policy**, not core material data.

Best setup now

Keep JSON as-is.

Inside Houdini, derive `bump_cap` from:

- material family
- finish

- condition
- maybe bump_type

For example:

```
if finish == polished and condition == clean:  
    bump_cap = 0.006  
else:  
    bump_cap = 0.02
```



Or slightly more refined:

```
if family == translucent:  
    bump_cap = 0.004  
elif finish == polished and condition == clean:  
    bump_cap = 0.006  
elif bump_type == cellular:  
    bump_cap = 0.018  
else:  
    bump_cap = 0.012
```



That is much better than stuffing another control into JSON.

What I recommend

In JSON

Keep only:

- bump_scale
- bump_type
- noise_scale
- variation_seed

In Houdini HDA

Add hidden/internal logic:

- bump_cap_default
- bump_cap_polished_clean
- bump_cap_translucent
- optional bump_cap_cellular

Then compute final internal cap.

When to add it to JSON later

Only add bump_cap to JSON if you later discover this specific need:

- you want explicit per-material override authored outside Houdini
- the generator must intentionally push some materials beyond the default family rules
- you want exact external reproducibility of all internal clamp decisions

That is an **advanced override**, not a V1 requirement.

If you ever add it, make it optional:

</> JSON



```
"procedural_parameters": {  
  "bump_scale": 0.3,  
  "bump_type": "stochastic",  
  "noise_scale": 1.0,  
  "variation_seed": 42,  
  "bump_cap_override": 0.008  
}
```

Then in Houdini:

- if override exists, use it

- otherwise use internal derived cap

Final answer

For now: **define** `bump_cap` in Houdini MaterialX/HDA, not in JSON.

That keeps the JSON cleaner and gives you centralized control over shader stability.

The right hierarchy is:

1. JSON says what kind of bump the material wants.
2. Houdini decides how far that is allowed to go safely.

That is the correct split.



Fri, Apr 24 at 4:03 PM



Stochastic bump results. I assume we need to decrease the bump height? Do we need to adjust fractal scales/octaves?

Thought for 16s >

Yes.

This is not "micro bump" yet. It is **large soft lobe deformation**.

The screenshot says two things immediately:

1. **bump height is too strong**
2. **noise scale is too large / frequency is too low**

Octaves are **not** the main problem right now.

What is wrong visually

Your stochastic branch is producing:

- broad melted blobs
- pooled highlights
- macro waviness

That means the noise is reading like **surface shape deformation**, not micro-surface breakup.

For stochastic bump, you want:

- tiny reflection breakup
- subtle highlight shimmer
- no obvious blob pattern from normal viewing distance

What to change first

1. Decrease bump height first

Yes. Hard.

Cut it down a lot.

If you currently have something like:

```
bump_scale = 0.01
```



test:

```
0.002
```



If still too strong:

```
0.001
```



If you also use `Mt1X Heighttonormal.scale`, keep that at `1.0` first and control strength only with `bump_scale`.

Do **not** boost both:

- `bump_scale`
- and `heighttonormal.scale`

at the same time.

That doubles the chaos.

2. Increase stochastic noise frequency

This is the second big fix.

Your noise is too broad.

The pattern scale is too big for stochastic micro-surface.

If you used my earlier coarse/fine example like:

```
coarse = noise_scale * 6  
fine   = noise_scale * 24
```



that is too low for the result you want here.

Try this instead:

```
coarse = noise_scale * 30  
fine   = noise_scale * 120
```



Or even:

```
coarse = noise_scale * 50  
fine   = noise_scale * 200
```



For stochastic bump, the frequencies should be much tighter than what looks good for variation or dirt.

Right now your bump reads like:

- medium form breakup

It should read like:

- micro surface

3. Reduce the fine branch weight if needed

Sometimes the fine branch makes the result noisy in a bad way, but here the bigger problem is the coarse branch being too broad.

Still, a good safe split is:

```
coarse_weight = 0.85  
fine_weight   = 0.15
```



instead of:

```
0.7 / 0.3
```



That keeps the result cleaner.

Are octaves the problem?

Not really.

Not yet.

Octaves mainly affect:

- richness
- fractal complexity
- how “layered” the noise feels

They do **not** fix the core issue if:

- height is too strong
- frequency is too low

So:

Keep octaves simple

Use:

coarse fractal

octaves = 2 or 3



fine fractal

octaves = 1 or 2



Do not go high on octaves for polished or semi-clean materials.

Too many octaves can make the result:

- crunchy
- busy
- fake

For stochastic micro bump, lower octave counts are usually safer.

Correct troubleshooting order

Do this in this order only.

Step 1

Cut bump height by about 5x to 10x.

Step 2

Increase frequency by about **5x to 10x**.

Step 3

Only then adjust octaves.

If you change all three at once, you will not know what fixed it.

What I would test right now

For your stochastic branch, test exactly this:

coarse branch

```
frequency = noise_scale * 40  
octaves = 2  
lacunarity = 2.0  
diminish = 0.5  
weight = 0.85
```



fine branch

```
frequency = noise_scale * 160  
octaves = 1  
lacunarity = 2.0  
diminish = 0.5  
weight = 0.15
```



bump strength

```
bump_scale = 0.0015  
bump_cap   = 0.004 to 0.006
```



That should already look far more believable.

Important material-specific point

The selected material in your screenshot is:

```
iron_brushed_scratched
```



That material should **not** use stochastic as the final branch.

It should use **directional**.

So if stochastic looks wrong here, that is not surprising.

You are testing the branch, which is fine, but do not judge the final iron material by stochastic behavior.

For `iron_brushed_scratched` :

- the final branch should be `directional`
 - stochastic should only be a weak secondary breakup layer
-

Practical rule for stochastic branch

Use stochastic for:

- soft matte breakup

- satin irregularity
- subtle polished imperfection
- rubber/plastic micro breakup

Do **not** expect it to carry:

- brushed grain
- scratches
- hammered pits

That belongs to other branches.

One more likely issue

If your Fractal3D output is already roughly zero-centered, and then you remap it again badly, you can accidentally amplify it too much.

For stochastic branch:

- keep the raw fractal nearly as-is
- blend the two layers
- multiply by small `bump_scale`
- clamp
- send to `Heighttonormal`

Do not aggressively remap contrast unless you really need it.

High contrast in height = ugly bump fast.

Blunt diagnosis

From the screenshot:

- **Yes**, lower bump height.
- **Yes**, raise fractal frequency a lot.
- **No**, octaves are not the first thing to fix.
- And for this material, the final answer is still **directional**, not stochastic.

My exact next move

Change only these first:

```
bump_scale:  divide by 5 to 10  
coarse frequency: multiply by 6  
fine frequency:  multiply by 6 to 8
```



Leave octaves low:

```
coarse octaves = 2  
fine octaves   = 1
```



That is the correct next test.

After that, show me:

- one grey clay material with stochastic bump only
- one polished metal
- one rough dielectric

That will expose whether the branch is actually under control.



ChatGPT can make mistakes. Check important info.